



## APPLICATION NOTE

# Measuring CMIS firmware update time with the Binho Supernova

AN0011

## Document information

|                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Keywords</b> | AN0011, Supernova, Pulsar, CMIS, OIF, CDB, firmware update, LPL, EPL, tWRITE, tCDBF, tBPC, Nmax, optical module, hold-off, I2CMCI, I3C, SPIMCI, transactions per block                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Abstract</b> | Updating firmware on an optical module is slow, and the obvious remedy is a faster management bus. Working through the CMIS access limits and the Table 10-4 timing parameters shows that the wire is not where the time goes: for a 1 MiB image the bus clock accounts for a small fraction of the total and the per-block hold-offs account for most of it. This application note derives the budget from the specification, states plainly which numbers are arithmetic and which would have to be measured, and supplies a command that times the access phases of a real module against the specification ceilings so that a host can find out which hold-offs its module actually takes. |

**Draft, revision 0.4.** The budget in Section 4 is arithmetic derived from the specification, and now carries all three management interfaces the specification and this series cover: I2CMCI, SPIMCI, and I3C as a Binho derivation. The `timing` output in Section 5 is a real run against firmware targets, not an optical module, and it turns out to measure the host path rather than the target. Section 7 says so plainly, reports what two adapter firmware revisions changed about it, and states what is outstanding. This revision is for internal review and is not for publication.

# 1 Introduction

I3C single data rate runs roughly 31 times faster than I2C Fast-mode on the wire. A host engineer facing a slow module firmware update reasonably expects a faster management bus to make it roughly 31 times quicker.

Working the arithmetic through the CMIS access limits gives a different answer, and the difference is actionable: the choice of which Common Data Block command carries the payload matters more than the choice of bus.

This note derives the budget, and supplies a way to find out where a specific module actually spends its time rather than where the specification permits it to.

**This document does not state a measured throughput.** Section 7 explains why, and what would have to be run to state one.

## 1.1 Scope

The document covers the arithmetic of a CMIS firmware download: the access length limits, the block sizes of the two Write Firmware Block commands, and the Table 10-4 hold-off parameters. It supplies a command that times the read, page change and Common Data Block access phases of a module against those ceilings.

It does **not** write firmware to a module, and the supplied utility has no command that does. It does not cover the firmware update state machine, image signing, or the Run and Commit sequence beyond noting their timing. AN0008 covers the register model and is a prerequisite.

## 1.2

**Applicable devices****Table 1.** Applicable devices

| Item                    | Role         | Basis                                                                |
|-------------------------|--------------|----------------------------------------------------------------------|
| Binho Supernova         | Host adapter | Exercised, Section 7                                                 |
| Binho Pulsar            | Host adapter | Not exercised                                                        |
| Any CMIS managed module | Target       | <b>Not exercised.</b> The measurement used an adapter in target mode |

**The budget in Section 4 is arithmetic, not measurement.** It is derived from OIF-CMIS-05.3 sections 5.2.2 and chapter 9 and from Table 10-4, and is reproducible with `python cmis_mci.py budget`. No row of it has been checked against a module.

## 1.3

**Related documents****Table 2.** Related documents

| Document      | Subject                                                                    |
|---------------|----------------------------------------------------------------------------|
| OIF-CMIS-05.3 | Sections 5.2.2.1 and 5.2.2.2, chapter 9 for the CDB, Table 10-4 for timing |
| AN0008        | Bringing up a CMIS optical module over I2C with the Binho Supernova        |
| AN0009        | Managing a CMIS optical module over I3C with the Binho Supernova           |
| AN0010        | Bringing up a CMIS optical module over SPI with the Binho Pulsar           |

## 2

**Required equipment and software****Table 3.** Required equipment

| Item                                                                                  |
|---------------------------------------------------------------------------------------|
| Binho Supernova or Binho Pulsar USB host adapter, for the <code>timing</code> command |
| A CMIS managed optical module, for the <code>timing</code> command                    |
| Nothing at all, for the <code>budget</code> command                                   |

```
pip install binhosupernova
python cmis_mci.py budget
```

### 3 The four facts the budget rests on

#### 3.1 A firmware block over the Local Payload is 116 bytes

The Common Data Block occupies Page 9Fh. Its Local Payload is 120 bytes, of which `Write Firmware Block LPL`, command `0103h`, spends 4 on the block offset. That leaves **116 bytes** of firmware per block.

The Extended Payload is pages A0h through AFh, 16 pages of 128 bytes, giving **2048 bytes** per block through `Write Firmware Block EPL`, command `0104h`.

#### 3.2 A WRITE carries at most 8 bytes

Section 5.2.2.2 states it directly: "A successful WRITE writes a sequence of up to eight given byte values". `FullPageReadSupported` raises `Nmax` to 128 for **reads**, and does nothing for writes.

The only escape is the note immediately following: "A WRITE with more than 8 bytes may be allowed when specified explicitly in chapter 7.8", which is the Common Data Block Extended Payload region.

So a 116 byte Local Payload block is  $\text{ceil}(116 / 8) = 15$  **separate write transactions**, each with its own address phase.

#### 3.3 The module then stops talking

**Table 4.** Hold-off parameters, CMIS Table 10-4 maxima

| Parameter | Maximum          | Applies after                                  |
|-----------|------------------|------------------------------------------------|
| tREAD     | 500 microseconds | A register read                                |
| tWRITE    | 10 ms            | A write to volatile memory                     |
| tWRITENV  | 80 ms            | A write to non-volatile memory                 |
| tBPC      | 10 ms            | A bank or page change                          |
| tCDBF     | 4960 ms          | A foreground CDB command. The default is 80 ms |
| tMgmtInit | 2000 ms          | Reset, until ModuleLowPwr is reached           |

These are maxima. A module may advertise anything up to them, and Section 5 is about finding out what a given module actually takes.

#### 3.4 Almost all CMIS memory is volatile

The specification states it outright, so **tWRITE at 10 ms is the common case**, not the optimistic one.

The hold-off is paid **once per block**, not once per transaction. That is the whole lever: a 2048 byte block pays it 512 times over a 1 MiB image, and a 116 byte block pays it 9040 times.

## 4

### The budget

```
python cmis_mci.py budget --image-kib 1024
```

Firmware update budget, 1024 KiB image, tWRITE 10 ms per block

| path                         | blocks | per block | bus     | hold-off | total    |
|------------------------------|--------|-----------|---------|----------|----------|
| I2CMCI 400 kHz, LPL 0103h    | 9040   | 15 txn    | 36.61 s | 90.40 s  | 127.01 s |
| I3C SDR 12.5 Mb/s, LPL 0103h | 9040   | 15 txn    | 1.17 s  | 90.40 s  | 91.57 s  |
| SPIMCI 1 MHz, LPL 0103h      | 9040   | 15 txn    | 15.19 s | 90.40 s  | 105.59 s |
| I2CMCI 400 kHz, EPL 0104h    | 512    | 16 txn    | 24.33 s | 5.12 s   | 29.45 s  |
| I3C SDR 12.5 Mb/s, EPL 0104h | 512    | 16 txn    | 0.78 s  | 5.12 s   | 5.90 s   |
| SPIMCI 1 MHz, EPL 0104h      | 512    | 1 txn     | 8.41 s  | 5.12 s   | 13.53 s  |
| SPIMCI 50 MHz, EPL 0104h     | 512    | 1 txn     | 0.17 s  | 5.12 s   | 5.29 s   |

Bus clock alone, 400 kHz to 12.5 Mb/s, about 31 times the wire speed: 1.39 times faster.

Payload size alone, LPL to EPL at 400 kHz: 4.3 times faster.

Both together: 21.5 times faster.

SPIMCI at 50 MHz, 125 times the I2C wire speed and one transaction per block: 24.0 times faster.

Read the first two rows against each other. Multiplying the wire speed by 31 removes 35 seconds. Changing which Common Data Block command carries the payload, at the original slow clock, removes 97.

#### 4.1

### Three management interfaces, and the same answer from all of them

The SPIMCI rows carry the comparison the whole series is built to make, because SPIMCI is the only other management interface CMIS actually specifies. Appendix B.3.7.1 frames every transaction as a four byte control word, **N** flow control bytes and up to **2048 bytes** of payload, and `MciSpeedConfiguration` at `00h:27.3-0` runs from 1 MHz to **50 MHz**. That is 125 times the I2C clock and 17 times the largest access the paged interfaces can make.

It buys 24.0 times against I2CMCI's 21.5. Two facts are worth separating out of that.

**At the speed a module actually starts at, SPIMCI is slower on the wire than I3C.** The initial `MciSpeedConfiguration` on exit from `MgmtInit` is always 1 MHz, and at 1 MHz the Extended Payload path spends 8.41 s on the wire against I3C's 0.78 s. The clock has to be raised deliberately, by a host that read the module's advertisement and wrote the field.

**What SPIMCI changes at any speed is the transaction count.** An Extended Payload block is 16 transactions on I2CMCI and on I3C, both of which cap an access at one page, and **one** on SPIMCI. That column never appears in a bandwidth figure, and Section 5 shows it is the one a host feels: every transaction costs a round trip through the host, and on the bench that round trip is about two milliseconds whatever the bus is doing.

So the three interfaces disagree by a factor of 125 on the wire and by a factor of 24 on the total, and they agree completely on where the time goes. A firmware update is not a bandwidth problem.

**Table 5.** What the budget model does and does not include

| Included                                                                 | Excluded                                      |
|--------------------------------------------------------------------------|-----------------------------------------------|
| Transactions per block, from the access length limits                    | Host USB latency, which is not a bus property |
| Bus time per transaction, as 4 framing bytes plus payload at 9 bits each | tCDBF, which varies by module and command     |
| tWRITE once per block                                                    | The Run Image and Commit Image steps          |
|                                                                          | Retries after a rejected access               |

**The bus time column is an estimate of wire occupancy, not a measured throughput.** It is deliberately crude, because its only job is to show that it is small next to the hold-off column. A more careful model of it would not change the conclusion, and a measured value would characterize the host's USB path as much as the bus.

## 5

### Measuring a real module

```
python cmis_mci.py timing
```

The command writes no firmware. It times the access phases an update is built from, a single byte read, an eight byte read at the write limit, a full page read where the module advertises one, a page change in both directions, and reaching the Common Data Block page, and prints each next to its Table 10-4 ceiling.

```
$ python cmis_mci.py timing
```

| phase                         | median   | max      | CMIS max |
|-------------------------------|----------|----------|----------|
| single byte read              | 2.058 ms | 3.400 ms | 0.5 ms   |
| 8 byte read, the WRITE limit  | 1.995 ms | 2.064 ms | -        |
| page change 00h to 01h to 00h | 7.988 ms | 9.019 ms | 20.0 ms  |
| select CDB page 9Fh           | 4.043 ms | 5.913 ms | 10.0 ms  |

One 116 byte LPL firmware block is 15 write accesses, about 29.9 ms of bus time, against a tWRITE hold-off of up to 10 ms that the module pays once per block.

**Read that as a measurement of the host, not of the target.** Every row is roughly two milliseconds per USB round trip: one access costs about 2 ms, the two register writes of a CDB page selection cost about 4, and the four writes of a page change in both directions cost about 8. A single byte read sits four times above tREAD, and an eight byte read is not slower than a one byte read, which is only possible if the bytes on the wire are not what is being timed.

The target in this run is an adapter in I2C target mode, which answers immediately and never holds the bus off. So what the numbers show is the floor the host path imposes before a module has contributed anything at all, and that floor is around 2 ms per access on this setup. Section 7.2 draws the conclusion.

What to look for in the result:

**A median far below the specification maximum** means the module is faster than it is permitted to be, and a host that waits the maximum after every write is leaving most of the update time unclaimed. This is the most valuable thing the measurement can tell you.

**A maximum close to the specification ceiling on a page change** means the module really does take tBPC sometimes. A host that assumes the median will fail intermittently, which is the worst failure mode to debug.

**A single byte read above tREAD** is host side or adapter side overhead, not module hold-off. tREAD is an abstraction over I2C clock stretching, and clock stretching is visible on a bus capture in a way that a host side timer cannot distinguish from anything else.

## 5.1

### The same measurement against an I3C CMIS target

The run above is over I2CMCI against an adapter in target mode. Repeating it over I3C against the CMIS target firmware of AN0009, which implements the register model rather than a flat memory, moves nothing that matters:

```
$ python cmis_mci.py timing --transport i3c --address 0x09 --push-pull
PUSH_PULL_2_5_MHZ_25_DC
```

| phase                         | median   | max      | CMIS max |
|-------------------------------|----------|----------|----------|
| single byte read              | 2.059 ms | 3.008 ms | 0.5 ms   |
| 8 byte read, the WRITE limit  | 2.062 ms | 2.760 ms | -        |
| page change 00h to 01h to 00h | 7.997 ms | 9.170 ms | 20.0 ms  |
| select CDB page 9Fh           | 3.998 ms | 4.120 ms | 10.0 ms  |

One 116 byte LPL firmware block is 15 write accesses, about 30.9 ms of bus time, against a tWRITE hold-off of up to 10 ms that the module pays once per block.

A 31 times faster bus, and a target that answers from firmware rather than from a flat arena, leave every row where it was. That is the same result the budget predicts, arrived at from the other direction.

## 5.2 Adapter firmware 4.4.0 against 4.5.0

Supernova firmware 4.5.0 raises the bulk transport limit and reworks the USB path, so it is the obvious candidate for lowering the floor Section 5 measures. It does not.

**Table 6.** The same four phases on two adapter firmware revisions, median of three runs

| Phase                         | 4.4.0    | 4.5.0    |
|-------------------------------|----------|----------|
| single byte read              | 2.006 ms | 2.036 ms |
| 8 byte read, the WRITE limit  | 2.003 ms | 2.036 ms |
| page change 00h to 01h to 00h | 7.993 ms | 8.061 ms |
| select CDB page 9Fh           | 3.987 ms | 4.035 ms |

The board was taken from 4.4.0 to 4.5.0 and back to 4.4.0 and forward again, and measured in each state, because a single comparison across a reflash cannot tell a firmware change from anything else that moved.

**The reason is the access sizes, and it is the note's own argument.** Every phase here is a one byte or an eight byte access, and this target advertises `Nmax 8`, so nothing in the measurement is large enough for a larger bulk buffer to carry. A transport improvement appears in a transfer big enough to fill it, and CMIS never issues one on the paged interfaces: 5.2.2.2 caps a WRITE at eight bytes and an access at one page. The interface that does issue one is SPIMCI, at 2048 bytes, and no SPIMCI target exists on this bench.

## 6 What to do with the finding

**For host software.** Query `Firmware Management Features`, CDB command `0041h`, once at the start of a download and prefer `Write Firmware Block EPL ( 0104h )` over `0103h` where the

module supports it. Do not assume either way.

**For module firmware.** Advertise hold-off times you can honor. tWRITE is a maximum, not a target, and the measurement in Section 5 is how a host discovers what you left unclaimed.

**For anyone specifying a new MCI binding.** SPIMCI already carries up to 2048 bytes in one transaction, which Section 4.1 prices, and I3C negotiates the same thing natively through SETMWL, SETMRL and GETMXDS. A binding that inherits I2CMCI's 8 byte write limit inherits this entire problem with it, and inherits it whatever clock it runs at.

**Budget the Run Image step separately.** CDB command 0109h takes as long as a full module reboot, "mainly governed by the tMgmtInit timing parameter", so allow roughly two seconds for it rather than a CDB timeout.

## 7

## Measured results

### 7.1

### Test configuration

**Table 7.** Test configuration, the I2CMCI run of Section 5

| Item            | Value                                                                                    |
|-----------------|------------------------------------------------------------------------------------------|
| Controller      | Binho Supernova, serial 1912DEB5...79FA , firmware 4.4.0, hardware revision C            |
| Target          | Binho Supernova, serial ED7BF8F3...F7BB , firmware 4.4.0, armed as an I2C target at 0x50 |
| Target contents | CMIS 5.3 image, the Appendix C 400G QSFP-DD worked example                               |
| SDK             | binhosupernova 4.2.0, Python 3.10.12                                                     |
| Host            | Ubuntu 22.04, x86-64                                                                     |
| Bus             | 3.3 V, 400 kHz                                                                           |
| Repeats         | 20 per phase, median and maximum reported                                                |
| Date            | 1 September 2026                                                                         |

**Table 8.** Test configuration, the I3C runs of Sections 5.1 and 5.2

| Item            | Value                                                                           |
|-----------------|---------------------------------------------------------------------------------|
| Controller      | Binho Supernova, serial 4F970418...1111 , firmware 4.4.0 and 4.5.0              |
| Target          | NXP FRDM-MCXXN947 running the AN0009 CMIS target firmware, dynamic address 0x09 |
| Target contents | CMIS 5.3 image, the Appendix C 400G QSFP-DD worked example                      |
| SDK             | binhosupernova 4.2.0, Python 3.14                                               |
| Host            | Arch Linux, x86-64                                                              |
| Bus             | 3.3 V, I3C SDR, push-pull 2.5 Mbit/s at 25% duty cycle, open drain 1 Mbit/s     |
| Repeats         | 20 per phase, median and maximum reported, three runs per firmware              |
| Date            | 3 September 2026                                                                |

The two configurations are different benches, different hosts and different Python versions. They are not compared against each other anywhere in this note, and only the second one is used for the firmware comparison in Section 5.2, where the host and the target are held fixed and the adapter firmware is the only thing that changes.

## 7.2

### Results

**Derived.** The budget in Section 4 is reproducible from the supplied utility and rests only on the specification citations in Section 3. It is arithmetic, and this document does not present it as anything else.

**Measured, and it measures the host.** The Section 5 run establishes a floor of about 2 ms per register access on this host, adapter and SDK, with a target that imposes no hold-off of its own. That number belongs to the USB path, not to the bus and not to a module: a single byte read exceeds tREAD fourfold, and an eight byte read is no slower than a one byte read, both of which are only possible if wire time is not the term that dominates.

This is the reason **no end to end throughput figure appears anywhere in this document**. A figure measured this way would characterize the host, and quoting it as a bus or module property would be wrong in a way that is hard to detect later.

It does, however, sharpen the note's own argument rather than weakening it. Section 4 argues that wire time is not where a firmware update goes. The measurement adds that host round trip cost is not where it goes either. Both are small next to a per-block hold-off, and both shrink for the same reason when a block gets larger: **fewer accesses and fewer blocks**. A 2 ms per access floor costs 30 ms per Local Payload block and 32 ms per Extended Payload block, but the Extended Payload block carries 17 times more firmware.

**Measured again, on a faster adapter, and it did not move.** Supernova firmware 4.5.0 leaves all four phases where 4.4.0 had them, within the run to run spread, with the host, the target and the bus held fixed. This is not a negative result about the firmware. It is the same finding as Section 4 reached by arithmetic: a transport gets faster in transfers large enough to fill it, and CMIS on a paged management interface never issues one, because 5.2.2.2 caps a WRITE at eight bytes and an access at one page. The measurement is bounded by the number of round trips, not by what each one can carry.

That is also why the transactions per block column in Section 4 is the one to read. SPIMCI carries an Extended Payload block in a single transaction where the paged interfaces need sixteen, and at about two milliseconds of host cost per transaction that is 2 ms against 32 ms per block before the bus has done anything at all.

**On hardware, against a module.** Nothing here. Outstanding at revision 0.4: the `timing` command against a real module, which is the only thing that establishes the hold-offs a module actually takes; a complete firmware download over both the Local Payload and Extended Payload paths, which is the only thing that would support an end to end figure; and any measurement at all on SPIMCI, for which no target exists on this bench. Every SPIMCI number in Section 4 is arithmetic from Appendix B.3 and is labelled as such.

Until that has been run, the claim this note supports is about **where the time is permitted to go**, and about what the host costs before a module contributes anything.

## 8

## Troubleshooting

**Table 9.** Troubleshooting

| Symptom                                                        | Probable cause and action                                                                           |
|----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>timing</code> reports a single byte read far above tREAD | Host or adapter overhead, not the module. Compare against a bus capture                             |
| Every phase is close to its ceiling                            | The module may be genuinely slow, or another controller is contending for the bus                   |
| The 94 byte read is absent from the output                     | The module does not advertise full page read, so <code>Nmax</code> stayed at 8                      |
| Medians vary widely between runs                               | Something else is using the USB path. Close other host tools and repeat                             |
| A page change exceeds twice tBPC                               | The module is taking longer than the specification permits. Capture the bus before blaming the host |

## 9

## Acronyms

**Table 10.** Acronyms

| Term | Meaning                                        |
|------|------------------------------------------------|
| CDB  | Common Data Block                              |
| CMIS | Common Management Interface Specification      |
| EPL  | Extended Payload, CDB pages A0h to AFh         |
| LPL  | Local Payload, in CDB page 9Fh                 |
| MCI  | Management Communication Interface             |
| Nmax | Maximum number of bytes in one register access |
| OIF  | Optical Internetworking Forum                  |
| SDR  | Single data rate                               |

## 10

## Note about the source code in the document

Example code shown in this document and supplied in the accompanying archive is provided for evaluation and reference. It is released under the MIT license, the text of which is included in the archive.

**Table 11.** Contents of the assets archive

| File                             | Description                                        |
|----------------------------------|----------------------------------------------------|
| <code>AN0011.md</code>           | Markdown source of this document                   |
| <code>assets/cmisi_mci.py</code> | Reference utility, shared by AN0008 through AN0011 |
| <code>assets/LICENSE</code>      | MIT license covering the example code              |

## 11

## Revision history

Table 12. Revision history

| Revision | Date             | Description                                                                                                                                                                                                                                                                                                                                                                          |
|----------|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0.1      | 1 September 2026 | Draft for internal review. Budget derived from the specification; no measurement taken.                                                                                                                                                                                                                                                                                              |
| 0.2      | 1 September 2026 | <code>timing</code> run against a Supernova I2C target fixture. Output pasted, and reported as a measurement of the host path rather than of a module.                                                                                                                                                                                                                               |
| 0.3      | 2 September 2026 | Renumbered from AN0010. No change to the measurements or the budget.                                                                                                                                                                                                                                                                                                                 |
| 0.4      | 3 September 2026 | Budget extended to SPIMCI, at its initial 1 MHz and at the 50 MHz top of <code>MciSpeedConfiguration</code> , with the transactions per block column called out as the one a host feels. Section 5 gains the same measurement over I3C against the AN0009 CMIS target, and a comparison of Supernova firmware 4.4.0 against 4.5.0 that finds no change and says why. Utility at 1.1. |

## Legal information

### Definitions

#### Draft

A document marked draft is under internal review and subject to formal approval. Binho Inc. makes no representation or warranty as to its accuracy or completeness and accepts no liability arising from its use.

### Disclaimers

#### Limited warranty and liability

Information in this document is believed to be accurate and reliable. Binho Inc. makes no representation or warranty, express or implied, as to its accuracy or completeness, accepts no liability for the consequences of its use, and takes no responsibility for content originating outside Binho Inc.

In no event shall Binho Inc. be liable for any indirect, incidental, punitive, special or consequential damages, including without limitation lost profits, lost savings, business interruption, or costs of removing or replacing any product, whether or not based on tort, warranty, breach of contract or any other legal theory.

Notwithstanding any damages a customer might incur, the aggregate and cumulative liability of Binho Inc. is limited in accordance with its terms and conditions of commercial sale.

#### Right to make changes

Binho Inc. reserves the right to change the information in this document, including specifications and product descriptions, at any time and without notice. This document supersedes all information supplied prior to its publication.

#### Suitability for use

Binho Inc. products are not designed, authorized or warranted for use in life support, life-critical or safety-critical systems or equipment, nor in applications where failure of a Binho Inc. product can reasonably be expected to result in personal injury, death, or severe property or environmental damage. Any such inclusion or use is at the customer's own risk and Binho Inc. accepts no liability for it.

### Applications

Applications described here are illustrative only. Binho Inc. makes no warranty that they are suitable for the specified use without further testing or modification. Customers are responsible for the design and operation of their own applications and products, for appropriate design and operating safeguards, and for determining

whether a Binho Inc. product is suitable and fit for their purpose. Binho Inc. accepts no liability for assistance with applications or customer product design.

### Third-party products and specifications

This document refers to products, boards, specifications and documentation supplied by third parties, which remain the responsibility of their respective owners. Binho Inc. makes no warranty regarding them and their behavior may change without notice. Register maps, protocol details and device identifiers reproduced here were observed on the specific hardware and firmware versions stated in this document and must be confirmed against the vendor's own documentation for the reader's part.

### Example code

Source code supplied with this document is provided as an example, for evaluation and reference. It is not qualified for production use, has not been developed under a functional safety or security process, and is licensed under the terms accompanying it in the distribution archive.

### Terms and conditions of commercial sale

Binho Inc. products are sold subject to the general terms and conditions of commercial sale published by Binho Inc., unless otherwise agreed in a valid written individual agreement, in which case only the terms of that agreement apply.

### Export control

This document and the items it describes may be subject to export control regulations. Export may require prior authorization from the competent authorities.

### Security

All products may be subject to unidentified vulnerabilities or may support security standards with known limitations. The customer is responsible for the design and operation of its applications and products throughout their lifecycle to reduce the effect of such vulnerabilities, and for compliance with all legal, regulatory and security requirements concerning its products.

### Trademarks

All referenced brands, product names, service names and trademarks are the property of their respective owners. **Binho** and the Binho logo are trademarks of Binho Inc. **CMIS** and **OIF** are trademarks or registered trademarks of the Optical Internetworking Forum. **I3C** and **MIPI** are trademarks or registered trademarks of MIPI Alliance, Inc.